

Licence 3 Informatique — Introduction au Big Data

MINI-PROJET

Traitement de données avec Apache Spark et MongoDB

Sujet A — Avis clients e-commerce

Nom : Rokhaya Ba

Date : 12 août 2026

Dépôt Git : https://github.com/Rokhayaba24/BigData_avis

1. Contexte et objectif

Ce mini-projet répond à la consigne du sujet A du cours Big Data : construire une chaîne de traitement complète permettant d'analyser un volume important d'avis clients e-commerce avec Apache Spark, puis de stocker les résultats agrégés dans une base NoSQL orientée documents (MongoDB).

Le dataset utilisé est « Amazon Fine Food Reviews », contenant 568 454 avis clients sur des produits alimentaires vendus sur Amazon. Chaque ligne comporte l'identifiant du produit (ProductId), la note attribuée (Score, de 1 à 5), le texte de l'avis (Text), ainsi qu'un horodatage Unix (Time) permettant de situer temporellement chaque avis.

L'objectif est de calculer trois indicateurs à partir de ces données brutes (note moyenne par produit, top produits, évolution mensuelle du volume d'avis), puis de les stocker sous forme de documents JSON exploitables sans traitement supplémentaire.

2. Choix techniques

Pourquoi Spark plutôt que pandas ?

Spark a été choisi car il permet de traiter les données de façon distribuée, c'est-à-dire en les répartissant sur plusieurs cœurs ou plusieurs machines. Pandas, à l'inverse, charge toutes les données en mémoire sur un seul cœur, ce qui devient impossible quand le volume de données dépasse la mémoire disponible. Même si les 568 454 lignes de ce projet auraient pu être traitées avec pandas, l'objectif du cours était d'apprendre à utiliser un outil conçu pour les très gros volumes de données, qui ne tiendraient plus en mémoire sur une seule machine.

Pourquoi MongoDB plutôt que Cassandra ?

MongoDB a été choisi car les résultats de ce projet sont naturellement organisés par produit : chaque produit a une note moyenne et un nombre d'avis. Ce type de donnée se représente très bien sous forme de document JSON, un objet par produit. Cassandra est plus adapté à des données qui arrivent en continu et en très grand volume, comme des mesures de capteurs ou des logs, ce qui n'est pas le cas ici. Le modèle document de MongoDB est donc plus simple et plus adapté à ce projet.

Le partitionnement

Lors du chargement du fichier CSV, Spark a créé une seule partition, alors que le fichier contient 568 454 lignes. Une seule partition signifie que Spark ne peut pas répartir le travail sur plusieurs cœurs du processeur, ce qui ralentit le traitement. Pour corriger cela, la fonction `repartition(4)` a été utilisée avant le nettoyage complet, afin de découper les données en 4 morceaux traités en parallèle. Ce choix de 4 partitions correspond aux ressources disponibles sur un ordinateur portable classique.

Environnement d'exécution

Le développement a été réalisé en local, dans un notebook Jupyter, avec PySpark 4.2.0 installé via pip. MongoDB a été exécuté dans un conteneur Docker (image officielle `mongo:7`), ce qui évite d'avoir à installer MongoDB directement sur la machine et facilite la reproductibilité de l'environnement.

3. Nettoyage des données

Le nettoyage a été réalisé avec l'API `DataFrame` de Spark, en trois étapes successives : suppression des valeurs manquantes sur les colonnes critiques (`ProductId`, `Score`, `Text`), suppression des doublons stricts, et filtrage des notes hors de l'échelle 1-5.

Sur le dataset réel (568 454 lignes), aucune ligne n'a été supprimée par ce pipeline : ni valeur manquante, ni doublon, ni note hors échelle n'ont été détectés. Ce résultat s'explique par la nature du dataset « Amazon Fine Food Reviews », largement utilisé en formation car déjà nettoyé en amont par ses auteurs.

Afin de vérifier que le pipeline de nettoyage fonctionne réellement (et non qu'il n'a simplement jamais été mis à l'épreuve), un test a été réalisé en injectant volontairement trois lignes corrompues (`ProductId` manquant, `Score` hors échelle, `Text` manquant) dans une copie du dataset. Le pipeline a bien supprimé les 3 lignes corrompues, confirmant sa correction fonctionnelle.

Un point technique rencontré lors du chargement : le fichier CSV contient des guillemets mal échappés dans certains textes d'avis, causant un décalage de colonnes lors du parsing par défaut. Ce problème a été résolu en activant les options `multiLine` et `escape=""` lors de la lecture.

4. Modélisation du schéma NoSQL

Schéma du document (collection `stats_produits`) :

```
{
  "_id": "B007JFMH8M",
  "note_moyenne": 4.58,
  "nombre_avis": 913,
  "est_top_produit": false
}
```

Chaque produit est représenté par un seul document contenant toutes ses statistiques. Ce choix a été fait car MongoDB ne demande pas de définir un schéma fixe à l'avance, contrairement à une base SQL classique.

Avec une base SQL, il faudrait créer une table avec des colonnes fixes (`produit_id`, `note_moyenne`, `nombre_avis`). Si on voulait ajouter une nouvelle statistique plus tard, comme une liste des mots les plus utilisés dans les avis d'un produit, il faudrait modifier la structure de la table (`ALTER TABLE`), ce qui peut être compliqué sur une base déjà remplie.

Avec MongoDB, on peut simplement ajouter un nouveau champ dans les documents sans rien casser, car chaque document peut avoir une structure légèrement différente. C'est pour cette flexibilité que MongoDB a été choisi plutôt qu'une base relationnelle classique pour stocker ces résultats.

Le champ `_id` utilise directement l'identifiant du produit (`ProductId`), ce qui évite de créer un identifiant supplémentaire et permet de retrouver rapidement les statistiques d'un produit précis. Une seconde collection, `evolution_avis`, stocke un document par mois avec le nombre d'avis correspondant.

5. Indicateurs calculés et résultats

Indicateur 1 : note moyenne par produit

Le calcul de la note moyenne par produit (regroupement par `ProductId`) a permis d'obtenir une statistique pour chaque produit du dataset, avec son nombre d'avis associé. Ce résultat sert de base aux deux autres indicateurs. Au total, 74 258 produits distincts ont été identifiés et stockés dans MongoDB.

Indicateur 2 : top produits

Le top 20 des produits les mieux notés (avec un minimum de 10 avis pour être éligible) montre exclusivement des produits ayant une note moyenne de 5.0, avec un nombre d'avis allant de 18 à 33. Ce seuil minimum de 10 avis a été fixé pour éviter qu'un produit avec très peu d'avis mais tous excellents ne fausse le classement par rapport à des produits ayant beaucoup plus d'avis.

ProductId	note_moyenne	nombre_avis
B005EL6VOY	5.0	33
B000RL53PE	5.0	33
B001IZ9ME6	5.0	31
B002900XGQ	5.0	30

Indicateur 3 : évolution du volume d'avis dans le temps

L'agrégation par mois montre que les premiers avis du dataset remontent à octobre 1999, avec un volume très faible au départ (1 à 8 avis par mois). Le nombre d'avis augmente progressivement au fil des années, ce qui reflète la croissance de l'activité e-commerce d'Amazon sur cette période. Par exemple, novembre 2002 comptabilise déjà 21 avis, contre seulement 2 avis en octobre 1999. Cette tendance croissante se poursuit sur l'ensemble de la période couverte par le dataset, avec un pic d'activité observé sur les derniers mois disponibles.

annee_mois	nombre_avis
1999-10	2
2000-01	8

2002-02	15
2002-11	21

6. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées et résolues au cours de ce projet, principalement liées à l'environnement d'exécution local sous Windows.

Panne du moteur Docker Desktop

En tout début de projet, Docker Desktop refusait de démarrer son moteur Linux (erreur 500 Internal Server Error sur l'API interne du moteur), empêchant le lancement de tout conteneur. Après plusieurs tentatives de dépannage (redémarrage de WSL2, redémarrage du service Docker), la solution a été de basculer temporairement vers une installation locale de PySpark et Jupyter via pip, le temps que Docker redevienne fonctionnel pour la partie MongoDB.

Parsing incorrect du fichier CSV

Le fichier source contient des guillemets mal échappés dans certains textes d'avis, ce qui provoquait un décalage de colonnes lors de la lecture par défaut (des valeurs de type texte se retrouvaient dans la colonne Score). Ce problème a été résolu en ajoutant les options `multiLine=True` et `escape=""` à la lecture du CSV.

Échecs de conversion de type (cast)

L'utilisation de `.cast("float")` sur la colonne Score provoquait une erreur bloquante (`CAST_INVALID_INPUT`) dès qu'une valeur résiduelle mal parsée était rencontrée. Le remplacement par `try_cast` (via `F.expr("try_cast(Score as float)")`) a permis de transformer ces valeurs invalides en NULL plutôt que de faire échouer l'ensemble du job, ces lignes étant ensuite naturellement exclues par le filtre sur l'échelle 1-5.

Instabilité de la JVM Spark en environnement local

À plusieurs reprises, la connexion entre le noyau Python (Jupyter) et la JVM Spark sous-jacente s'est interrompue (`Py4JNetworkError`, `ConnectionResetError`), notamment après des calculs lourds sur le dataset complet sans configuration mémoire adaptée. La solution a consisté à limiter explicitement la mémoire du driver (`spark.driver.memory = 2g`), à travailler sur un échantillon réduit (5 %) pendant la phase de mise au point du pipeline, puis à ne repasser sur le dataset complet qu'une fois le code validé — une pratique méthodologique recommandée en développement Big Data local.

Erreur de communication du worker Python sous Windows

Lors de la création manuelle d'un petit DataFrame de test (via `spark.createDataFrame`), une erreur « Python worker failed to connect back » est apparue, liée à la résolution réseau locale sous Windows. Elle a été résolue en forçant explicitement l'adresse du driver Spark à l'adresse locale (`spark.driver.host` et `spark.driver.bindAddress` à `127.0.0.1`).

7. Conclusion et pistes d'amélioration

Ce projet a permis de mettre en œuvre une chaîne de traitement Big Data complète, depuis l'ingestion d'un fichier CSV volumineux jusqu'au stockage de résultats agrégés dans une base NoSQL, en passant par le nettoyage et le calcul d'indicateurs avec Apache Spark. Les trois indicateurs demandés (note moyenne par produit, top produits, évolution temporelle) ont été calculés avec succès sur l'ensemble des 568 454 avis du dataset, puis stockés dans MongoDB sous forme de documents exploitables directement, sans traitement supplémentaire.

Au-delà du résultat technique, ce projet a surtout permis de manipuler concrètement les concepts fondamentaux de Spark (évaluation paresseuse, transformations vs actions, partitionnement) ainsi que la logique de modélisation orientée documents propre au NoSQL, en la comparant explicitement à l'approche relationnelle classique.

Plusieurs pistes d'amélioration pourraient être explorées avec plus de temps :

- Ajouter une analyse de sentiment basique sur le texte des avis (Text), par exemple via un comptage de mots positifs/négatifs, pour enrichir les documents MongoDB d'un indicateur supplémentaire.
- Mettre en place un flux de données en continu avec Kafka et Spark Structured Streaming, pour simuler l'arrivée d'avis en temps réel plutôt qu'un traitement batch unique.
- Ajouter des tests automatisés (au-delà du test de robustesse manuel réalisé) pour valider systématiquement chaque étape du pipeline.
- Déployer le dashboard de visualisation (bonus) avec Streamlit pour une consultation interactive des résultats stockés dans MongoDB.
- Automatiser l'ensemble du pipeline via un orchestrateur comme Apache Airflow, pour un déclenchement périodique plutôt que manuel.

Annexes

Dépôt Git

https://github.com/Rokhayaba24/BigData_avis

Extrait de code — pipeline de nettoyage

```
df_full_clean = df.dropna(subset=["ProductId", "Score", "Text"]) \
    .dropDuplicates() \
    .withColumn("Score", F.expr("try_cast(Score as float)")) \
    .filter((F.col("Score") >= 1) & (F.col("Score") <= 5)) \
    .repartition(4) \
    .cache()
```